Semantic Caching with Federated Similarity Learning for LLM Serving

PRIVACY-PRESERVING TRAINING FOR COST-OPTIMIZED LLM INFERENCE

Name: Pojesh Kumar R
Reg: 22BAI1373

Guide: Dr. Kumar R - 50304

CONTENTS

- Problem Statement
- Problem Background
- Literature Survey
- Existing Solutions and Gaps
- Proposed Solution
- System Architecture
- Solution Details
- Project Scope
- Timeline and Milestones
- Tools and Technologies

PROBLEM STATEMENT

- **High deployment costs**: LLM inference is expensive at production scale, especially when there's large query volumes.

- **Cache similarity**: Research analyzing 27k ChatGPT conversations reveals that 31% of queries are semantically similar to previous queries, but existing cache systems capture only a small fraction due to rigid similarity requirements.

- **Misclassification**: Fixed-threshold similarity methods often misclassify queries and fail to adapt across diverse domains.

- **Privacy concerns**: Server-side caching requires sending user queries to central servers, raising risks of data exposure and breaches. No existing framework enables privacy-preserving, distributed training for semantic caching.

PROBLEM BACKGROUND

- **LLM API Growth**: Large Language Models like GPT and Claude power millions of applications including chatbots, search engines, and virtual assistants through API-based serving.

- **Semantic Caching**: Semantic caching stores query-response pairs and retrieves cached results for semantically similar queries, avoiding redundant API calls.

- **Federated Learning**: Federated Learning enables distributed model training across multiple clients without centralizing user data, preserving privacy.

- **Similarity-Based Retrieval**: Modern embedding models (MPNet, BERT) convert text to dense vectors, enabling semantic similarity computation using cosine distance.

- **Production Deployment**: Real-world LLM applications require cost reduction and latency optimization while maintaining accuracy and user privacy.

LITERATURE SURVEY

S.No.	Author	Approach	Key Contribution	Gap Identified
1	Haqiq et al., 2025	MinCache: Hybrid Cache System for Efficient Chatbots	Three-tier hierarchy: exact-resemblance-semantic; 4.5× speedup vs GPTCache	MinHash approximation misses nuanced semantics; threshold tuning complexity
2	Gill et al., 2025	MeanCache: User-Centric Semantic Caching	Federated learning for similarity models, contextual queries support	No adaptive thresholds; fixed similarity metrics
3	Gao et al., 2025	Online Context Caching for Distributed LLM Serving	KV cache placement + request scheduling in distributed systems	Doesn't address semantic similarity decisions
4	Jiang et al., 2024	LongLLMLingua: Prompt Compression via Information Density	Question-aware coarse-to-fine compression, position bias mitigation	Limited to compression only, no caching coordination
5	Li et al. 2024	Semantic Caching for Automated Chat Services	SCALM with vector quantization; semantic clustering; token savings	Semantic analysis overhead and no adaptive mechanisms

LITERATURE SURVEY

S.No.	Author	Approach	Key Contribution	Gap Identified
6	Liu, et al., 2024	Lost in the Middle: How LLMs Use Long Contexts	Position bias analysis in transformer models	Affects compression but not caching
7	Regmi & Pun, 2024	GPT Semantic Cache: Reducing LLM Costs and Latency	Redis-based semantic caching with HNSW ANN	Fixed threshold lacks adaptability; multi-turn conversations poorly handled
8	Jiang et al., 2024	LongLLMLingua: Prompt Compression via Information Density	Question-aware coarse-to-fine compression, position bias mitigation	Limited to compression only, no caching coordination
9	Li et al., 2024	Context-based Semantic Caching for LLM Applications	Context embedding + attention weighting + temporal decay; improved multi-turn accuracy	Increased memory footprint; context processing overhead; optimal window size difficult to determine
10	Peng et al., 2024	Yarn: Efficient Context Window Extension	Attention with Linear Biases; extends context to 128K tokens; efficient interpolation	Training required for specific models; doesn't reduce KV memory footprint

LITERATURE SURVEY

S.No.	Author	Approach	Key Contribution	Gap Identified
11	Zheng et al., 2024	SGLang: Efficient Execution of Structured Language Model Programs	Smart scheduling framework; Optimized tensor scheduling; prefix caching integration	Requires structured input format; complex pipeline hard to debug
12	Zha et al., 2023	Online optimization for semantic cache hit prediction	Bayesian optimization for threshold tuning; joint optimization of cache and routing	Doesn't account for task-specific heterogeneity; limited to single query type; manual threshold initialization
13	Zhai & Chen, 2023	Cached Transformers with Differentiable Memory	Gated Recurrent Cache + meta-learner, accuracy gain on vision/language tasks	Compression overhead; meta-learning adds latency; not evaluated on large-scale LLM inference
14	Zhu et al., 2023	Semantic Caching for LLM Cost Reduction	Embedding-based cache lookup with similarity thresholds	Fixed threshold τ=0.8, no adaptation
15	Xiao et al., 2023	Heavy-Hitter Oracle for KV Cache Compression	Identifies heavy-hitter attention; 20-50% KV reduction	Attention pattern dependent; fails on dense attention; requires offline analysis per model

EXISTING SOLUTIONS AND GAPS

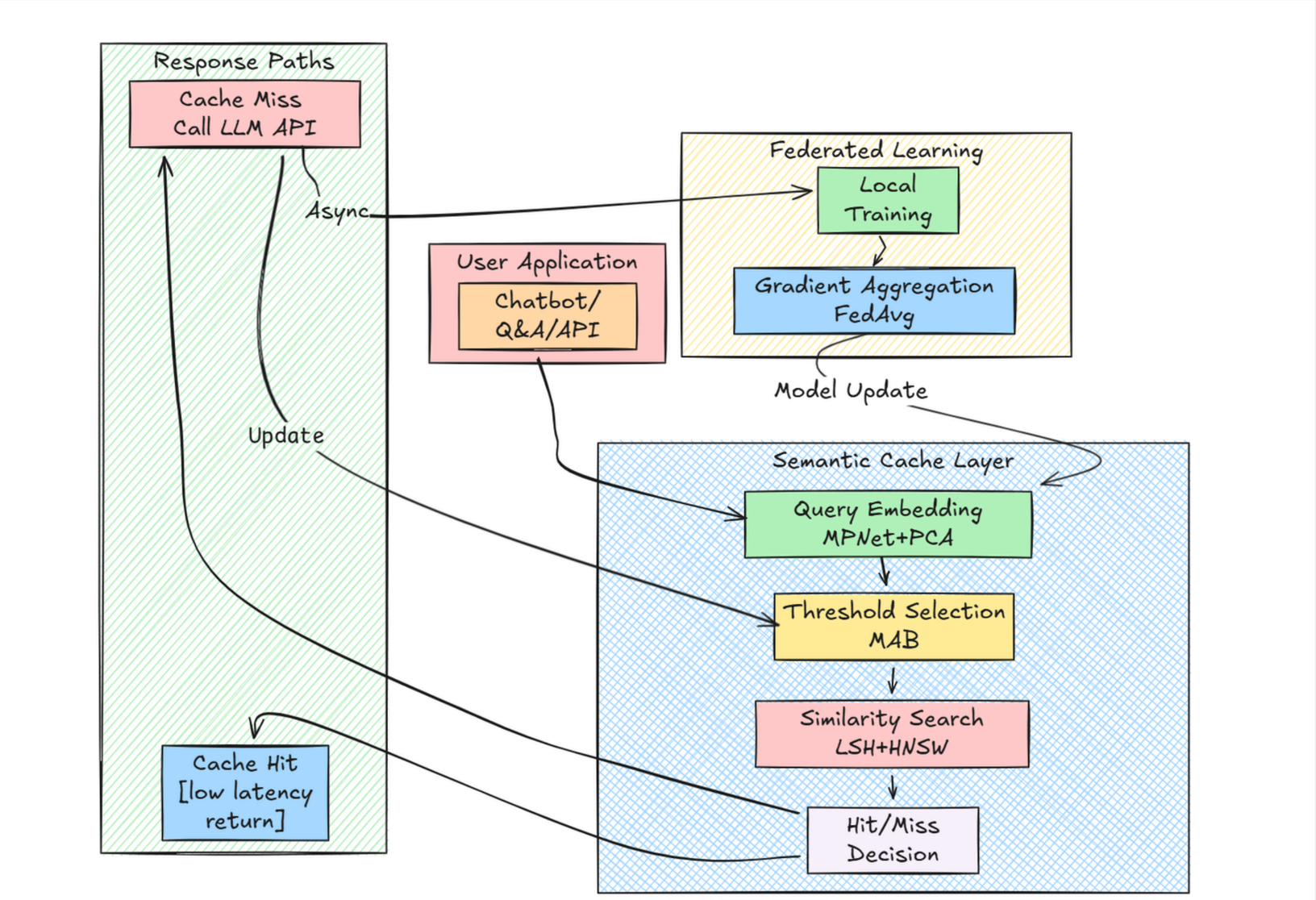
Solution	Hit Rate	Precision	Key Limitation
Exact Match	<5%	100%	Fails on paraphrases
GPTCache	40%	52%	Fixed threshold, privacy issues
MeanCache	45%	72%	No adaptive thresholding
SCALM	50%	65%	Chat services only
GPT Semantic Cache	68.80%	97%+	Centralized process

- **Privacy:** Centralized solutions achieve hit rates but violate user privacy by storing queries on servers.
- **Semantic cache**: All solutions use either no semantic matching or fixed thresholds that can't automatically adjust between domains.
- **Reliability Gap**: Decentralized solutions suffer from dangerously low precision (GPTCache: 52%, SCALM: 65%), leading to frequent "false hits" where the cache serves incorrect answers, whereas high reliability is currently only achievable through centralized processing.

PROPOSED SOLUTION

Component	Function
Federated Contrastive Learning	Trains embedding models across distributed user devices without sending raw queries to a central server. Each client learns to distinguish semantically similar queries from near-duplicates using contrastive and ranking loss, then only shares model weights for aggregation.
Contextual Multi-Armed Bandit	Dynamically selects optimal similarity thresholds per query based on context like domain, query length, and time of day. Uses Beta distributions to track success/failure rates for each context-threshold pair and continuously adapts from user feedback.
LSH-based Hierarchical Indexing	Combines Locality-Sensitive Hashing with Hierarchical Navigable Small World graphs to enable sub-linear search across cached queries. Quickly retrieves candidate neighbors using hash tables, then refines results using a multi-layer graph structure for accurate nearest-neighbor matching.

SYSTEM ARCHITECTURE



SOLUTION DETAILS

- **User Application**: The process begins when the Chatbot or API sends a user's query to the Semantic Cache Layer.
- **Query Embedding**: The system converts the text query into a compressed vector representation using the MPNet model and PCA.
- **Threshold Selection**: The Multi-Armed Bandit (MAB) analyzes the query context to dynamically select the optimal similarity threshold.
- **Similarity Search**: The system searches the database for the closest matching past queries using efficient LSH and HNSW algorithms.
- **Hit/Miss Decision**: The system compares the search result's similarity score against the selected threshold to determine if a match exists.
- **Cache Hit**: If a valid match is found, the stored response is returned to the user with minimum latency.
- **Cache Miss**: If no match is found, the system forwards the query to the external LLM API to generate a new response.
- **Update**: The new response is saved to the cache, and the MAB model is updated with feedback on the decision.
- **Federated Learning**: In the background, the system trains locally on user data and aggregates improvements via FedAvg to protect privacy.
- **Model Update**: The globally improved model weights are sent back to the Semantic Cache Layer to enhance future embedding accuracy.

PROJECT SCOPE

IN SCOPE:

- Semantic cache system with federated learning
- MAB-based adaptive thresholding and HNSW indexing integration
- Evaluation on ShareGPT and MS MARCO datasets
- FastAPI-based REST API server with Grafana monitoring

OUT OF SCOPE:

- Custom LLM training from scratch; use pretrained
- Support for non-English languages; focusing on English queries only
- Integration with proprietary LLM APIs; use of open source models only
- Real-time multi-user chat with session management

TIMELINE AND MILESTONE

Task	Deliverables
Literature survey	Literature survey completed
Data acquisition and processing	Clean data
MPNet embedding, HNSW/LSH indexing	Scalable vector search module
Federated Learning (FedAvg), MAB Thresholding	Privacy-preserving adaptive module
System integration, FastAPI development	End-to-end semantic cache service
Comparison with baselines	Performance metrics & ablation study
Deployment, documentation, paper	Working API + Research Paper

TOOLS AND TECHNOLOGIES

Category	Technology
Langauge	Python 3.10.11
Deep Learning Framework	PyTorch 2.1.0 with CUDA 11.8
Embedding Models	MPNET, BERT
Federated Learning	Flower (flwr) 1.6.0
In-Memory Cache	Redis with LRU eviction
Backend framework	FastAPI
Monitoring	Prometheus, Grafana

REFERENCES

1. *K. Haqiq, "MinCache: A hybrid cache system for efficient chatbots with hierarchical embedding matching and LLM," in Proc. Future Gener. Comput. Syst., vol. 165, pp. 107-120, 2025.*
2. *W. Gill, A. Elidrisi, et al., "MeanCache: User-Centric Semantic Caching for LLM Web Services," arXiv preprint arXiv:2403.02694, 2024.*
3. *X. Gao, et al., "Online Context Caching for Distributed Large Language Models Serving," in Proc. IEEE 39th Int. Parallel and Distrib. Process. Symp. (IPDPS), 2025.*
4. *H. Jiang, Q. Wu, et al., "LongLLMLingua: Accelerating and Enhancing LLMs in Long Context via Prompt Compression," arXiv preprint arXiv:2310.06839, 2023.*
5. *Z. Li, et al., "SCALM: Towards Semantic Caching for Automated Chat Services with Large Language Models," in Proc. IEEE/ACM Int. Symp. on Quality of Service (IWQoS), 2024.*
6. *N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, et al., "Lost in the Middle: How Language Models Use Long Contexts," Transactions of the Association for Computational Linguistics, vol. 12, pp. 157-173, 2024.*
7. *S. Regmi and C. P. Pun, "GPT Semantic Cache: Reducing LLM Costs and Latency via Semantic Embedding Caching," arXiv preprint arXiv:2411.05276, 2024*
8. *Y. Jiang, et al., "LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models," in Proc. EMNLP, 2023.*
9. *H. Li, et al., "Context-based Semantic Caching for LLM Applications," in Proc. IEEE Int. Conf. on Web Services (ICWS), 2024.*
10. *B. Peng, et al., "YaRN: Efficient Context Window Extension of Large Language Models," arXiv preprint arXiv:2309.00071, 2023.]*
11. *L. Zheng, et al., "SGLang: Efficient Execution of Structured Language Model Programs," arXiv preprint arXiv:2312.07104, 2023.*
12. *R. Zha, et al., "Online Optimization for Semantic Cache Hit Prediction," in Proc. IEEE Int. Conf. on Big Data (Big Data), pp. 690-699, 2023.*
13. *S. Zhai and Z. Chen, "Cached Transformers with Differentiable Memory," in Proc. IEEE/CVF Conf. on Comput. Vis. and Pattern Recognit. (CVPR), 2023.*
14. *F. Zhu, et al., "Semantic Caching for Large Language Model Cost Reduction," in Proc. IEEE Global Commun. Conf. (GLOBECOM), 2023.*
15. *G. Xiao, et al., "Efficient Streaming Language Models with Attention Sinks," arXiv preprint arXiv:2309.17453, 2023*

Thank you